

Q1. What is the syntax and semantics of the following Java statement: `int x = 5 + 3;`?

Syntax:

Syntax refers to the set of grammatical rules that define how symbols, keywords, and operators may be legally combined to form a valid statement in the language. The syntax of `int x = 5 + 3;` is defined by Java's grammar rules for a variable declaration with initialization. It consists of the following components, in order:

- `int` — a type specifier (keyword) declaring the data type.
- `x` — an identifier (the variable name).
- `=` — the assignment operator.
- `5 + 3` — an expression, itself built from two integer literals joined by the addition operator `+`.
- `;` — the statement terminator, required by Java grammar.

This can be represented by a simplified grammar rule:

```
<Declaration> ::= <Type> <Identifier> '=' <Expression> ';'
<Expression>  ::= <Literal> '+' <Literal>
```

Semantics:

Semantics refers to the meaning associated with a syntactically valid statement — what happens when it is executed. The semantics of this statement are:

- A memory location capable of holding an integer value is reserved and bound to the name `x` (declaration semantics).
- The expression `5 + 3` is evaluated according to the semantics of the `+` operator for integer operands, producing the value 8.
- The resulting value 8 is stored into the memory location for `x` (assignment semantics).

In short: the syntax is the correct textual form of the statement, while the semantics is the actual effect — here, that after execution the variable `x` holds the integer value 8.

Q2. Identify lexical tokens in a simple Java program.

A lexical token is the smallest individual unit of a program that has meaning to the compiler's lexical analyzer (scanner). Consider the following simple Java program:

```
public class Sum {
    public static void main(String[] args) {
        int a = 10;
```

```

        int b = 20;
        int sum = a + b;
        System.out.println(sum);
    }
}

```

The lexical analyzer would break this program into tokens belonging to the following categories:

- Keywords: public, class, static, void, int
- Identifiers: Sum, main, args, a, b, sum, System, out, println
- Literals: 10, 20 (integer literals)
- Operators: = (assignment), + (addition)
- Separators / Punctuation: {, }, (,), [,], ;, .
- Data type: String (a class type used here as a type specifier)

Each of these tokens is recognized individually by the lexer and passed on to the parser, which checks whether the sequence of tokens conforms to Java's grammar (syntax analysis).

Q3. Modify a sample program to demonstrate the use of grammar rules in Java.

Grammar rules define how tokens may be combined into valid statements and larger constructs (expressions, blocks, methods, classes). Below, a simple program is modified to explicitly show several grammar rules in action: a class declaration rule, a method declaration rule, a conditional statement rule, and an expression rule.

```

public class GrammarDemo {
    public static void main(String[] args) {
        int marks = 85;           // <VariableDeclaration>

        // <IfStatement> ::= "if" "(" <Expression> ")" <Block> "else" <Block>
        if (marks >= 40) {
            System.out.println("Result: Pass");
        } else {
            System.out.println("Result: Fail");
        }
    }
}

```

The grammar rules demonstrated here include:

- `<VariableDeclaration> ::= <Type> <Identifier> "=" <Expression> ";"` — used for `int marks = 85;`

- `<RelationalExpression> ::= <Identifier> ">=" <Literal>` — used for the condition marks `>= 40`
- `<IfStatement> ::= "if" "(" <Expression> ")" <Block> "else" <Block>` — used for the entire if-else construct
- `<MethodCall> ::= <Identifier> "." <Identifier> "(" <Arguments> ")" ";"` — used for `System.out.println(...);`

This shows how a compiler's parser applies grammar production rules recursively to validate and structure a program made of nested statements and expressions.